# MLNN Final Project

March 9, 2026

**Machine Learning & Neural Networks Final Project**

**4.5.1 Defining the Problem and Assembling the Dataset**

The first stage of the machine learning workflow is to clearly define the prediction problem and identify the dataset that will be used. This involves determining the input data, the output to be predicted, and the type of machine learning task being addressed (1).

The dataset I will use in this project is the Reuters newswire dataset, which is available through the TensorFlow Keras datasets library. The dataset contains thousands of reputable news articles collected from the Reuters news agency. Each article has been labelled with the topic that the article discusses. These topic labels mean that the data will be suitable for use as testing and training data within the grander process of supervised machine learning (2).

The input data will be the text of news articles contained in the Reuters dataset. Since neural networks cannot process raw text directly, the text must first be converted into a numerical representation. Each article is therefore encoded as a string of integers representing words from a predefined vocabulary (words are assigned numbers so that the text can be read as integers). **These strings form the input variable X**, which will be provided to the neural network during training and prediction. Neural networks process numerical data structures called tensors, so converting the text into numeric form allows the model to learn patterns from the data (3) *Note: Scalars have 0 dimensions, vectors have 1, matrices have 2 and tensors have 3 or more.*

The goal of the model is to predict the topic category of each news article. Each article in the dataset is associated with a label representing the topic that the article belongs to. **These labels form the target variable Y**. During training, the neural network receives a news article encoded as integers as input and attempts to predict the correct topic label associated with that article. ***This type of task is a multiclass classification problem.*** It is **multiclass classification** because each input examle belongs to one of several possible categories, and the model must determine which category is the most likely. In the Reuters dataset, each news article belongs to exactly one topic category, meaning that the model must assign the correct category based on the content of the article. This is as opposed to binary classification where there are only two categorical options, or the numerical outputs seen in vector regression.

In this first stage of the machine learning workflow *two key assumptions* are made.

1) The first assumption is that the topic of a news article can be predicted from the words contained in the article. In other words, it is assumed that the input text contains enough information for the model to determine the correct topic. Without this assumption, the ML model and NN might be ineffective.

2) The second assumption is that the dataset contains enough labelled examples for the NN to learn this relationship between the text and the category label. Machine learning models learn patterns that exist within the training data, so if the dataset does not contain meaningful relationships between inputs and outputs within the training data, the model may fail to achieve useful predictive performance and be ineffective or inaccurate.

These assumptions will only be validated after the model has been trained and evaluated later in the ML workflow.

Clearly defining the prediction task, the dataset, the input data, and the expected outputs ensures that the machine learning problem is well specified before model development begins.

#### 4.5.2 Choosing a Measure of Success

After outlining the input, output and problem classification type, the next step in the workflow is to *determine how the success of the model will be quantified*. This is necessary because it provides an objective way to evaluate the performance of the model and to determine whether the model is improving during the training process. Without a defined metric it would not be possible to compare different models, loss functions, or determine whether the model is learning useful patterns from the data (1).

The task in this project is to classify news articles into their correct topic category. Because each article belongs to one category from several possible classes, the problem is a multiclass classification task. For problems of this type, a commonly used evaluation metric is classification accuracy. Accuracy measures the proportion of predictions made by the model that are correct when compared with the true labels in the dataset ***(accuracy = number of correct predictions / total number of predictions)***. Additional documentation on classification metrics is also provided in the TensorFlow documentation, which describes accuracy as the fraction of predictions that are correctly classified (4). Note: the higher the score of accuracy, the more samples were correctly predicted out of the samples which were tested.

This metric was selected based on guidance from the Deep Learning with Python workflow, which recommends accuracy as a suitable evaluation metric for balanced classification problems. We know in advance that the reuters dataset has been used extensively in machine learning research and has been defined as reasonably balanced and appropriated for ML tasks (1). Alternative and common metrics for success of a ML task would be MSE (mean squared error) and precision though these are more suitable for tasks which output numerical values and tasks which contain one rarely occurring class within a multiclass dataset respectively.

Although accuracy is used as the primary measure of success when evaluating the model, neural networks do not optimise accuracy directly during training. Instead, the training process minimises a loss function, which measures the difference between the predicted output of the model and the true target value. For multiclass classification problems, the commonly used loss function is categorical cross-entropy. This loss function compares the multiclass classification probability by the neural network with the correct class label and produces a numerical value representing the prediction error with a lower score being better(5).

During the training iterations the neural network will minimise the categorical cross-entropy loss while accuracy will be monitored as the main performance metric and increase if the NN is performing correctly. Monitoring both values provides insight into how well the model is learning from the data and previous iterations, helping identify issues such as underfitting or overfitting as training progresses. By defining *accuracy as the primary measure of success* and using *categorical*

*cross-entropy as the loss function*, the performance of the model can be evaluated consistently throughout the machine learning workflow.

### 4.5.3 Deciding on an Evaluation Protocol

Once suitable metrics for success have been decided, now we need to decide how the model will be evaluated during development. An evaluation protocol describes how the dataset being used for training and testing will be divided so that the performance of the model can be measured in a reliable way. This step is important because a ML model must be tested on data that it has not seen during training. If a model were evaluated only on the data used to train it, **the results could appear artificially good because the model may simply memorise the training examples rather than learn patterns that generalise to new data**.

There are several evaluation protocols which are commonly used. These include ***hold-out validation, k-fold cross-validation, and repeated cross-validation***. When a dataset contains a sufficiently large number of samples, the **hold-out validation** approach is often considered the most practical method because it is simple to implement and computationally efficient (1). This method involves splitting the dataset into separate subsets that are used for training, validation, and testing.

In this project the hold-out validation approach will be used as there are over 10,000 labelled examples to be split into training, testing and validation, with the training and testing sets being predefined. The training data will be used to train the neural network, while the test data will be reserved for evaluating the final model after training has been completed. During model development, a portion of the training data will be set aside as a validation set. *The validation set allows the performance of the model to be monitored while training is taking place and helps identify whether the model is improving.* It is the set against which the model is tested between iterations under current parameters.

Each subset of the data serves a different purpose. The training set is used to update the parameters of the neural network as it learns from the data through iterations. The validation set is used to monitor model performance during development and to help guide decisions about model design and parameter adjustmen. The test set is used only once the final model has been selected in order to provide an unbiased estimate of how well the model performs as a whole on previously unseen data (6). This separation helps ensure that the reported results **reflect the model's ability to generalise rather than its ability to remember the training examples.**

Using a structured evaluation protocol such as hold-out validation is considered good practice in machine learning because it allows different model configurations to be compared fairly and helps prevent overly optimistic estimates of model performance (7). Using the same subset of data for training, testing or validation would skew results because the neural network would simply remember the data it had been trained on or had seen before. This would give an unrealistic impression of how well the model will perform on new, unseen data.

### 4.5.4 Preparing the Data

Before a neural network can be trained, the data must be prepared in a form that the model can process. Neural networks cannot work directly with raw text. Instead, the text must be converted into numerical values and arranged in tensors so that mathematical operations can be performed during training as explained in Chollet (1). The goal of this stage is therefore to transform both the input data and the labels into a numerical format that can be used by the neural network. *Note: Tensors are more appropriate because they can represent* ***multi-dimensional data****, allowing*

*neural networks to process many features and many samples at the same time. Scalars, vectors, and matrices are limited to fewer dimensions, while tensors provide the flexible structure needed for complex machine learning computations.*

The news articles in the Reuters newswire dataset are **stored as sequences of integers**; integers which have been assigned to each word which appears in all the articles. We will be selecting the most commonly occurring words in order to use them for out neural network and machine learning model. With integers assigned to a word in a predefined vocabulary, the text can be processed numerically by the neural network. However, ***these sequences vary in length and cannot be used directly as input to a Dense neural network layer as every neuron would expect the same number of input values each time it runs***.

For this reason, the sequences must be converted into fixed-length vectors through a process known as **vectorisation**. In this project the vocabulary is limited to the 10,000 most frequently occurring words in order to keep the input representation uniform and reduce noise caused by outlying and rare words. To vectorise the data we will create a vector of length 10,000, where each of the most common 10,000 words occupies a position. If a word within the 10,000 appears in the article, it's position will be switched to a value of 1, otherwise it will remain 0. This turns the article into a uniform format which the Dense neural network can process.

The first step is to load the dataset using the TensorFlow Keras datasets module.

[2]:
```
#import the Reuters dataset from TensorFlow keras
from tensorflow.keras.datasets import reuters

#load the dataset
#specify the number of words from the predefined vocabulary which will be used
#the dataset is returned as the training and testing subsets
(train_data, train_labels), (test_data, test_labels) = reuters.
 ↪load_data(num_words=10000)
```

Now that the tensorflow dataset from reuters has been imported, we can process the data to input to out neural network.

As discussed above, the dataset stores each article as a list of numbers, where each number represents a word from the vocabulary. Because Dense neural network layers require inputs of the same size, each article must be converted into a fixed-length list of 10,000 indices, where positions corresponding to words in the article are marked with 1 and all other positions remain 0.

[3]:
```
#import numpy library as standard
import numpy as np

#function to convert sequences of word indices into binary vector indices
#sequences contains the articles and dimension represents the size of the␣
 ↪vocabulary
def vectorise_sequences(sequences, dimension=10000):

    #create 2D array of zeros by default
    #number of rows = number of articles i.e. length of sequences
    #number of columns = vocabulary size i.e. 10,000
```

```
    results = np.zeros((len(sequences), dimension))

    #loop through each article in the dataset
    for i, sequence in enumerate(sequences):

        #for the current article, set the indices for present vocabulary to 1
        #if words at indices [1,4] are contained in article for example, then␣
 ↪the array will read [1,0,0,1]
        results[i, sequence] = 1.0

    #return completed matrix
    return results

#apply the vectorisation function to both training and test data present in the␣
 ↪reuters dataset
x_train = vectorise_sequences(train_data)
x_test = vectorise_sequences(test_data)
```

This process converts each news article into a fixed-length vector where each position indicates whether a particular word from the vocabulary appears in the article. This representation allows the neural network to process the input data efficiently and learn patterns in the words that are associated with different article categories.

The category labels must also be converted into a suitable numerical format. One-hot encoding is used so that the correct category is represented in the same vector format as the probabilities predicted by the neural network. In this format, each category label becomes a vector of zeros with a single value of 1 marking the correct category.

During training, the neural network predicts how likely the article belongs to each category, giving each one a **value between 0 and 1**. These predictions are compared with the correct category so the model can see how wrong it was and adjust itself to make better predictions next time.

[4]:
```
#keras model tools
from tensorflow import keras
from tensorflow.keras import layers
#for L2 Weight Regularisation
from tensorflow.keras import regularizers

#import a utility function to perform one-hot encoding
from tensorflow.keras.utils import to_categorical

#convert integer labels into one-hot encoded vectors
y_train = to_categorical(train_labels)
y_test = to_categorical(test_labels)
```

This code converts the article categories into a format the neural network can understand. Each category number is turned into a list of zeros with a single 1 marking the correct category.

This is necessary because the neural network predicts a probability for every possible category, rather than just giving one category number. By converting the correct answers into the same

vector format, the model can compare its predictions with the correct category during training and measure how accurate it is.

After these steps, both the article data and the category labels are stored as numerical tensors, which means they are now in the correct format to be used when training the neural network in the next stage.

[5]:
```
print("Training data shape:", x_train.shape)
print("Test data shape:", x_test.shape)

print("Training labels shape:", y_train.shape)
print("Test labels shape:", y_test.shape)
```

```
Training data shape: (8982, 10000)
Test data shape: (2246, 10000)
Training labels shape: (8982, 46)
Test labels shape: (2246, 46)
```

These outputs confirm that the input data and labels have been successfully converted into tensors with consistent dimensions. If this were not the case, the neural network would not be able to operate as it requires the dimensions to be the same for mathematical operations to be carried out.

The data is now in a suitable format for training the neural network in the next stage of the workflow.

### 4.5.5 Developing a Model that Performs Better than a Baseline

After preparing the dataset and converting the input data and category labels into tensors, the next step is to construct an initial neural network model. The objective at this stage is to develop a simple model that performs better than a naive baseline.

*A naive baseline is a very simple prediction method, such as random guessing or always choosing the most common class, used as a minimum performance benchmark that a machine learning model should exceed.*

Achieving performance above this baseline demonstrates that the dataset contains meaningful patterns that the model can learn, and therefore, the model shows that the input data contains meaningful information for predicting the correct label (1).

In multiclass-classification problems it is common practice to use a softmax activation function in the final layer, because softmax converts the network's raw outputs into a probability distribution across all possible classes. The corresponding loss function is categorical cross-entropy *(explained above)*, which measures how different the predicted probability distribution is from the correct category label. This combination of softmax and categorical cross-entropy is widely used for multiclass classification tasks in neural networks (1), and is better suited than other means, as explained above in section 4.5.1.

The model is implemented using the Keras Sequential Model, which allows neural networks to be constructed as a simple linear stack of layers (8). Dense layers are used to perform fully connected transformations of the input data, enabling the network to learn relationships between the numerical input vectors and the output categories.

A Dropout layer is also included. Dropout works by randomly turning off some neurons during

each training step, so the network cannot rely too heavily on any single pathway it has previously used. This forces the model to learn more general patterns in the data, which reduces the likelihood of memorising the training set and therefore helps prevent overfitting (9).

At this stage, the network architecture is intentionally kept simple. The goal is not yet to achieve the highest possible accuracy, but rather to establish a baseline neural network that demonstrates predictive capability. Once this baseline model is trained and evaluated, further experimentation with model capacity, regularisation, and hyperparameters can be carried out in subsequent stages of the workflow.

[6]:
```
#initialise a Sequential neural network - a linear stack of layers where each␣
↪layer feeds into the next
model = keras.Sequential()

#add the first Dense layer
#64 neurons allows model to learn patterns while maintaining its simplicity
#ReLU (Rectified Linear Unit) introduces non-linearity, allowing the network to␣
↪learn complex patterns
model.add(layers.Dense(64, activation='relu', input_shape=(10000,)))

#Dropout layer to reduce overfitting
#0.5 means that 50% of neurons are randomly switched off during each training␣
↪step
#this forces the network to learn more general patterns instead of memorising␣
↪training data
#can be adjusted later with other hyper parameters
model.add(layers.Dropout(0.5))

#output layer
#46 neurons correspond to the 46 possible news topic categories in the dataset
#softmax activation converts the outputs into probabilities that sum to 1
#the class label with the highest probability will be the model's predicted␣
↪category
model.add(layers.Dense(46, activation='softmax'))

#compile the model so it is ready for training
#rmsprop is the optimisation algorithm that updates the network weights while␣
↪learning is taking place
#categorical_crossentropy measures the difference between predicted␣
↪probabilities and correct labels
#accuracy is used as a metric to monitor how many predictions are correct out␣
↪of all predictionsmade
model.compile(
    optimizer='rmsprop',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
#print a summary of the model architecture
#this displays each layer, its output shape, and the number of trainable␣
↪parameters (weights, biases etc.)
model.summary()
```

```
C:\Users\Alex\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:106:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.
Please use WSL2 or the TensorFlow-DirectML plugin.
```

**Model: "sequential"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| dense (Dense) | (None, 64) | 640,064 |
| dropout (Dropout) | (None, 64) | 0 |
| dense_1 (Dense) | (None, 46) | 2,990 |

**Total params:** 643,054 (2.45 MB)

**Trainable params:** 643,054 (2.45 MB)

**Non-trainable params:** 0 (0.00 B)

The model summary confirms that the neural network has been successfully constructed. The output shape **(None, 64)** in the first Dense layer indicates that each input article is transformed into a vector of 64 features, while the value **None** represents a flexible batch size, meaning the network can process any number of articles at once during training or prediction. The summary also shows the number of trainable parameters in each layer, which represent the weights and biases that will be adjusted during training *automatically by the optimizer (rmsprop)* in order to minimise the loss and improve prediction accuracy. *Note: the 2.45 MB indicates the approximate amount of memory required to store all the trainable parameters (weights and biases) of the neural network.*

### 4.5.6 Developing a Model that Overfits

The purpose of this stage is to determine whether the baseline model has sufficient capacity to learn the structure of the dataset. In machine learning, the ideal model lies between underfitting *(a model that is too simple to capture patterns in the data)* and overfitting *(a model that memorises the training data but performs poorly on unseen data)*. In order to find this balance, it is useful to deliberately construct a model that overfits.

A model can be encouraged to overfit by increasing its capacity, which means allowing it to learn more complex relationships within the training data. This can be achieved by increasing the number of neurons in hidden layers, adding additional layers, or training the model for more epochs. When a model begins to overfit, the training loss continues to decrease while the validation loss begins to increase, **indicating that the network is memorising the training data rather than generalising to unseen samples.**

In this experiment, the architecture of the baseline model is expanded by increasing the number of neurons in the hidden layer. This increases the number of trainable parameters and therefore the model's ability to fit the training data. The objective is to observe whether the model begins to overfit when its capacity is increased.

[7]:
```
#construct a larger neural network to test model capacity
model_overfit = keras.Sequential()

#double the number of neurons to increase model complexity
model_overfit.add(layers.Dense(128, activation='relu', input_shape=(10000,)))

#remove dropout to reduce regularisation and encourage overfitting
model_overfit.add(layers.Dense(46, activation='softmax'))

#compile the model
model_overfit.compile(
    optimizer='rmsprop',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

#print architecture summary
model_overfit.summary()
```

```
Model: "sequential_1"


 Layer (type)                    Output Shape                  ␣
↪Param #

 dense_2 (Dense)                 (None, 128)                   ␣
↪1,280,128
```

```
dense_3 (Dense)                      (None, 46)                        ␣
↪5,934
```

**Total params:** 1,286,062 (4.91 MB)

**Trainable params:** 1,286,062 (4.91 MB)

**Non-trainable params:** 0 (0.00 B)

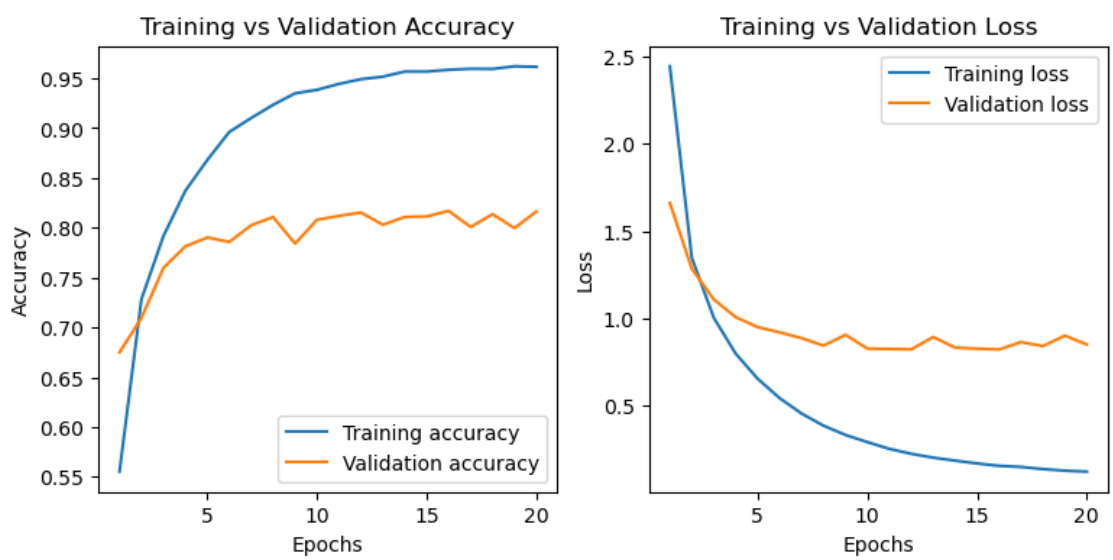
[8]:
```
history_overfit = model_overfit.fit(
    x_train,
    y_train,
    epochs=20,
    batch_size=512,
    validation_split=0.2
)
```

```
Epoch 1/20
15/15              4s 128ms/step -
accuracy: 0.5553 - loss: 2.4447 - val_accuracy: 0.6750 - val_loss: 1.6618
Epoch 2/20
15/15              1s 52ms/step -
accuracy: 0.7283 - loss: 1.3483 - val_accuracy: 0.7095 - val_loss: 1.2857
Epoch 3/20
15/15              1s 55ms/step -
accuracy: 0.7915 - loss: 1.0054 - val_accuracy: 0.7596 - val_loss: 1.1098
Epoch 4/20
15/15              1s 84ms/step -
accuracy: 0.8370 - loss: 0.7995 - val_accuracy: 0.7813 - val_loss: 1.0086
Epoch 5/20
15/15              1s 70ms/step -
accuracy: 0.8679 - loss: 0.6559 - val_accuracy: 0.7902 - val_loss: 0.9511
Epoch 6/20
15/15              1s 55ms/step -
accuracy: 0.8960 - loss: 0.5449 - val_accuracy: 0.7858 - val_loss: 0.9217
Epoch 7/20
15/15              1s 53ms/step -
accuracy: 0.9101 - loss: 0.4567 - val_accuracy: 0.8024 - val_loss: 0.8882
Epoch 8/20
15/15              1s 58ms/step -
accuracy: 0.9233 - loss: 0.3871 - val_accuracy: 0.8108 - val_loss: 0.8453
Epoch 9/20
15/15              1s 56ms/step -
accuracy: 0.9349 - loss: 0.3340 - val_accuracy: 0.7841 - val_loss: 0.9072
```

```
Epoch 10/20
15/15              1s 50ms/step -
accuracy: 0.9383 - loss: 0.2929 - val_accuracy: 0.8080 - val_loss: 0.8285
Epoch 11/20
15/15              1s 51ms/step -
accuracy: 0.9443 - loss: 0.2542 - val_accuracy: 0.8119 - val_loss: 0.8262
Epoch 12/20
15/15              1s 51ms/step -
accuracy: 0.9491 - loss: 0.2261 - val_accuracy: 0.8152 - val_loss: 0.8243
Epoch 13/20
15/15              1s 59ms/step -
accuracy: 0.9516 - loss: 0.2039 - val_accuracy: 0.8030 - val_loss: 0.8947
Epoch 14/20
15/15              1s 53ms/step -
accuracy: 0.9567 - loss: 0.1868 - val_accuracy: 0.8108 - val_loss: 0.8336
Epoch 15/20
15/15              1s 67ms/step -
accuracy: 0.9567 - loss: 0.1707 - val_accuracy: 0.8114 - val_loss: 0.8282
Epoch 16/20
15/15              1s 52ms/step -
accuracy: 0.9585 - loss: 0.1571 - val_accuracy: 0.8169 - val_loss: 0.8232
Epoch 17/20
15/15              1s 50ms/step -
accuracy: 0.9595 - loss: 0.1507 - val_accuracy: 0.8008 - val_loss: 0.8656
Epoch 18/20
15/15              1s 50ms/step -
accuracy: 0.9594 - loss: 0.1385 - val_accuracy: 0.8136 - val_loss: 0.8429
Epoch 19/20
15/15              1s 50ms/step -
accuracy: 0.9620 - loss: 0.1294 - val_accuracy: 0.7997 - val_loss: 0.9025
Epoch 20/20
15/15              1s 54ms/step -
accuracy: 0.9614 - loss: 0.1240 - val_accuracy: 0.8164 - val_loss: 0.8519
```

[9]:
```python
import matplotlib.pyplot as plt

#extract values from training history
acc = history_overfit.history['accuracy']
val_acc = history_overfit.history['val_accuracy']
loss = history_overfit.history['loss']
val_loss = history_overfit.history['val_loss']

epochs = range(1, len(acc) + 1)

#create two plots side by side
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9,4))
```

```
#accuracy plot
ax1.plot(epochs, acc, label='Training accuracy')
ax1.plot(epochs, val_acc, label='Validation accuracy')
ax1.set_title('Training vs Validation Accuracy')
ax1.set_xlabel('Epochs')
ax1.set_ylabel('Accuracy')
ax1.legend()

#loss plot
ax2.plot(epochs, loss, label='Training loss')
ax2.plot(epochs, val_loss, label='Validation loss')
ax2.set_title('Training vs Validation Loss')
ax2.set_xlabel('Epochs')
ax2.set_ylabel('Loss')
ax2.legend()

plt.show()
```




The training and validation accuracy and loss graphs above illustrate how the model's performance evolves over the 20 training epochs. The **training accuracy** increases steadily from approximately **0.56 in the first epoch to around 0.96 by epoch 20**, while the **training loss** decreases consistently from about **2.46 to roughly 0.12**. This trend shows that the model is progressively fitting the training data and learning the patterns present in the dataset.

However, the validation curves display a different pattern. The validation accuracy improves during the early stages of training, **increasing from approximately 0.64 to around 0.81**, but **after roughly 9 epochs it begins to stabilise**, fluctuating within a narrow range for the remainder of training. At the same time, the validation loss initially decreases but then begins to increase gradually after around **epoch 10**, rising from roughly **0.82 to about 0.87** by the final epoch.

This increase in validation loss suggests that the model is beginning to memorise the training data, because although it continues fitting the training data more closely, its prediction error on unseen validation data begins to worsen, indicating that the patterns being learned are becoming specific to the training set rather than generalisable.

The divergence between the training and validation curves visible in the graphs indicates the onset of **overfitting**. While the model continues to improve its performance on the training dataset, these improvements are not reflected in the validation results, **which represent unseen data**. Instead, the network begins to learn increasingly specific patterns in the training data such as noise or irregularities which doesn't improve the model as a whole. As a result, training accuracy continues to increase while validation performance remains relatively stable and validation loss begins to rise. This behaviour demonstrates that the model has sufficient capacity to fit the training data and confirms that increasing model capacity can lead to overfitting.

The optimal points for accuracy and loss occur at slightly different epochs because they measure different aspects of model performance. In this case, the optimal epoch for validation accuracy appears to occur around epoch 9 or 10, while the lowest validation loss occurs later at approximately epoch 12.

### 4.5.7 Regularizing the Model and Tuning Hyperparameters

Having demonstrated in the previous section that increasing the capacity of the neural network by increasing the number of active neurons leads to overfitting, the next stage of the workflow is to apply regularisation techniques and tune the model's hyperparameters in order to improve generalisation performance.

The training curves from the overfitting experiment showed that while training accuracy continued to improve, validation performance stabilised and validation loss began to increase after approximately epoch 12, indicating that the model had begun to memorise patterns specific to the training data rather than learning generalisable features. To address this behaviour, a series of controlled experiments will be conducted to determine a more balanced model configuration.

First, the strength of dropout regularisation will be adjusted to examine how different dropout rates affect generalisation.

Secondly, L2 weight regularisation will be introduced to constrain the magnitude of network weights and encourage simpler representations. *L2 weight regularisation penalises large weights during training, encouraging the model to learn simpler patterns and reducing overfitting.*

***A third experiment about neurons amount in the hidden dense layer had to be aborted because my kernel was fried and I was running out of time***

#### 4.5.7.1 Experiment 1 - Adjusting the Dropout Regularisation Strength

The first experiment examines whether the strength of dropout regularisation used in the baseline model is appropriate. In the baseline architecture a dropout rate of **0.5** was used, meaning that half of the neurons in the hidden layer are randomly deactivated during each training step. While dropout helps reduce overfitting by preventing the network from relying too heavily on individual neurons, an excessively high dropout rate can also remove too much information during training and reduce the model's learning capacity.

To investigate this, the dropout rate will be adjusted while keeping the rest of the model architecture unchanged. The hidden layer will continue to use **64 neurons with ReLU activation**, and

the output layer will remain a **46-unit softmax layer** for multiclass classification. The only modification will be altering the dropout rate and setting it to **0.3, 0.5, 0.8** in order to evaluate the strengths and weaknesses shown from a range of dropouts.

The model will be trained using the same training configuration as previous experiments, including the same number of epochs, batch size, and validation split. Keeping these conditions constant ensures that any changes in training behaviour can be attributed to the adjusted dropout rate.

After training, the training and validation accuracy and loss will be plotted again. These results will be compared with those observed in the previous section in order to determine whether the adjusted dropout rate produces a better balance between fitting the training data and maintaining stable validation performance.

[10]:
```
#dropout values to test
dropout_rates = [0.1, 0.3, 0.5, 0.8]

#empty data structure to store training histories
histories_dropout = {}

#loop for each dropout value
for rate in dropout_rates:

    print(f"\nTraining model with dropout rate = {rate}")

    model_dropout = keras.Sequential()

    #first Dense layer as before
    model_dropout.add(layers.Dense(64, activation='relu', input_shape=(10000,)))

    #dropout layer as before with varying rate in each loop
    model_dropout.add(layers.Dropout(rate))

    #output layer
    model_dropout.add(layers.Dense(46, activation='softmax'))

    #compile model
    model_dropout.compile(
        optimizer='rmsprop',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    #train model
    history = model_dropout.fit(
        x_train,
        y_train,
        epochs=20,
        batch_size=512,
```

```
        validation_split=0.2
    )

    #store results
    histories_dropout[rate] = history
```

```
Training model with dropout rate = 0.1
Epoch 1/20
15/15              2s 55ms/step -
accuracy: 0.5116 - loss: 2.8065 - val_accuracy: 0.6021 - val_loss: 2.0411
Epoch 2/20
15/15              0s 26ms/step -
accuracy: 0.6668 - loss: 1.7442 - val_accuracy: 0.6978 - val_loss: 1.5348
Epoch 3/20
15/15              1s 31ms/step -
accuracy: 0.7350 - loss: 1.3183 - val_accuracy: 0.7251 - val_loss: 1.3094
Epoch 4/20
15/15              0s 24ms/step -
accuracy: 0.7733 - loss: 1.0816 - val_accuracy: 0.7585 - val_loss: 1.1605
Epoch 5/20
15/15              1s 29ms/step -
accuracy: 0.8134 - loss: 0.9122 - val_accuracy: 0.7602 - val_loss: 1.0789
Epoch 6/20
15/15              1s 27ms/step -
accuracy: 0.8374 - loss: 0.7848 - val_accuracy: 0.7813 - val_loss: 1.0091
Epoch 7/20
15/15              0s 24ms/step -
accuracy: 0.8622 - loss: 0.6836 - val_accuracy: 0.7841 - val_loss: 0.9646
Epoch 8/20
15/15              0s 23ms/step -
accuracy: 0.8786 - loss: 0.6040 - val_accuracy: 0.7891 - val_loss: 0.9317
Epoch 9/20
15/15              0s 24ms/step -
accuracy: 0.8924 - loss: 0.5361 - val_accuracy: 0.7991 - val_loss: 0.9065
Epoch 10/20
15/15              1s 36ms/step -
accuracy: 0.9045 - loss: 0.4782 - val_accuracy: 0.8063 - val_loss: 0.8777
Epoch 11/20
15/15              0s 27ms/step -
accuracy: 0.9168 - loss: 0.4261 - val_accuracy: 0.8147 - val_loss: 0.8566
Epoch 12/20
15/15              0s 25ms/step -
accuracy: 0.9233 - loss: 0.3846 - val_accuracy: 0.8164 - val_loss: 0.8474
Epoch 13/20
15/15              1s 34ms/step -
accuracy: 0.9324 - loss: 0.3431 - val_accuracy: 0.8180 - val_loss: 0.8272
Epoch 14/20
```

```
15/15              0s 26ms/step -
accuracy: 0.9364 - loss: 0.3123 - val_accuracy: 0.8186 - val_loss: 0.8314
Epoch 15/20
15/15              1s 29ms/step -
accuracy: 0.9422 - loss: 0.2824 - val_accuracy: 0.8208 - val_loss: 0.8217
Epoch 16/20
15/15              1s 28ms/step -
accuracy: 0.9434 - loss: 0.2577 - val_accuracy: 0.8197 - val_loss: 0.8195
Epoch 17/20
15/15              0s 27ms/step -
accuracy: 0.9496 - loss: 0.2362 - val_accuracy: 0.8219 - val_loss: 0.8223
Epoch 18/20
15/15              0s 26ms/step -
accuracy: 0.9527 - loss: 0.2215 - val_accuracy: 0.8230 - val_loss: 0.8268
Epoch 19/20
15/15              0s 25ms/step -
accuracy: 0.9503 - loss: 0.2061 - val_accuracy: 0.8236 - val_loss: 0.8091
Epoch 20/20
15/15              1s 30ms/step -
accuracy: 0.9546 - loss: 0.1906 - val_accuracy: 0.8186 - val_loss: 0.8326

Training model with dropout rate = 0.3
Epoch 1/20
15/15              2s 47ms/step -
accuracy: 0.5009 - loss: 2.9950 - val_accuracy: 0.6639 - val_loss: 2.1413
Epoch 2/20
15/15              0s 25ms/step -
accuracy: 0.6518 - loss: 1.8990 - val_accuracy: 0.6817 - val_loss: 1.6004
Epoch 3/20
15/15              1s 33ms/step -
accuracy: 0.7042 - loss: 1.4607 - val_accuracy: 0.7201 - val_loss: 1.3479
Epoch 4/20
15/15              1s 33ms/step -
accuracy: 0.7524 - loss: 1.2080 - val_accuracy: 0.7557 - val_loss: 1.2080
Epoch 5/20
15/15              0s 27ms/step -
accuracy: 0.7809 - loss: 1.0467 - val_accuracy: 0.7674 - val_loss: 1.1174
Epoch 6/20
15/15              1s 33ms/step -
accuracy: 0.8099 - loss: 0.9201 - val_accuracy: 0.7769 - val_loss: 1.0529
Epoch 7/20
15/15              1s 36ms/step -
accuracy: 0.8301 - loss: 0.8222 - val_accuracy: 0.7819 - val_loss: 1.0013
Epoch 8/20
15/15              1s 34ms/step -
accuracy: 0.8461 - loss: 0.7466 - val_accuracy: 0.7947 - val_loss: 0.9571
Epoch 9/20
15/15              1s 39ms/step -
```

```
accuracy: 0.8637 - loss: 0.6599 - val_accuracy: 0.7902 - val_loss: 0.9421
Epoch 10/20
15/15            1s 35ms/step -
accuracy: 0.8731 - loss: 0.6051 - val_accuracy: 0.8036 - val_loss: 0.9054
Epoch 11/20
15/15            1s 34ms/step -
accuracy: 0.8885 - loss: 0.5459 - val_accuracy: 0.8086 - val_loss: 0.8955
Epoch 12/20
15/15            1s 35ms/step -
accuracy: 0.8976 - loss: 0.5055 - val_accuracy: 0.8130 - val_loss: 0.8676
Epoch 13/20
15/15            1s 37ms/step -
accuracy: 0.9047 - loss: 0.4591 - val_accuracy: 0.8130 - val_loss: 0.8583
Epoch 14/20
15/15            1s 34ms/step -
accuracy: 0.9164 - loss: 0.4240 - val_accuracy: 0.8152 - val_loss: 0.8470
Epoch 15/20
15/15            1s 33ms/step -
accuracy: 0.9183 - loss: 0.3940 - val_accuracy: 0.8164 - val_loss: 0.8272
Epoch 16/20
15/15            1s 34ms/step -
accuracy: 0.9247 - loss: 0.3616 - val_accuracy: 0.8158 - val_loss: 0.8327
Epoch 17/20
15/15            1s 36ms/step -
accuracy: 0.9308 - loss: 0.3395 - val_accuracy: 0.8164 - val_loss: 0.8314
Epoch 18/20
15/15            1s 38ms/step -
accuracy: 0.9344 - loss: 0.3060 - val_accuracy: 0.8158 - val_loss: 0.8375
Epoch 19/20
15/15            1s 40ms/step -
accuracy: 0.9379 - loss: 0.2940 - val_accuracy: 0.8191 - val_loss: 0.8348
Epoch 20/20
15/15            1s 38ms/step -
accuracy: 0.9415 - loss: 0.2679 - val_accuracy: 0.8214 - val_loss: 0.8197

Training model with dropout rate = 0.5
Epoch 1/20
15/15            2s 71ms/step -
accuracy: 0.4842 - loss: 2.8524 - val_accuracy: 0.5799 - val_loss: 2.0505
Epoch 2/20
15/15            1s 37ms/step -
accuracy: 0.6161 - loss: 1.8999 - val_accuracy: 0.6822 - val_loss: 1.5977
Epoch 3/20
15/15            1s 37ms/step -
accuracy: 0.6804 - loss: 1.5180 - val_accuracy: 0.6989 - val_loss: 1.3656
Epoch 4/20
15/15            1s 36ms/step -
accuracy: 0.7187 - loss: 1.3032 - val_accuracy: 0.7412 - val_loss: 1.2283
```

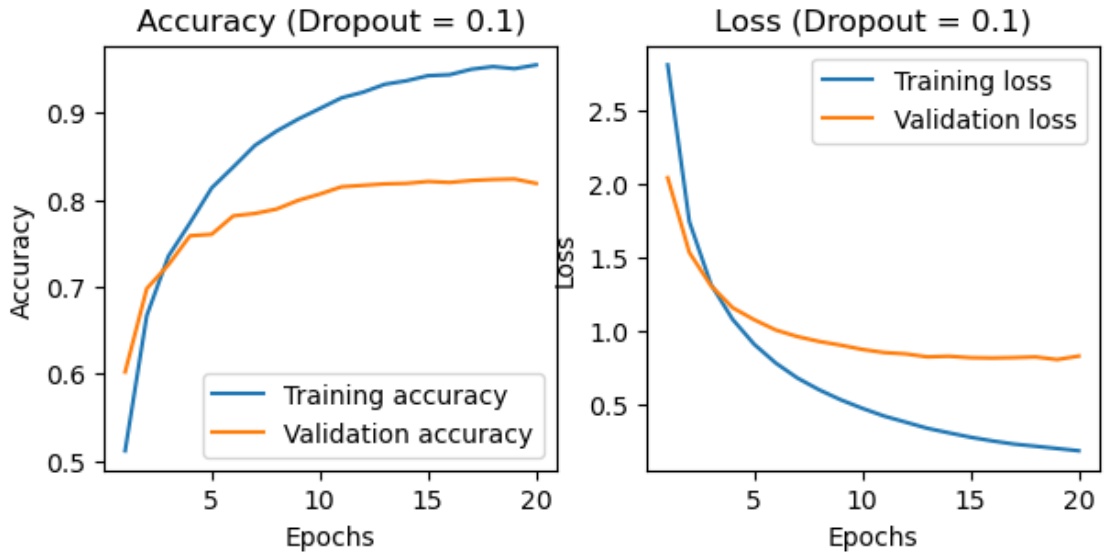
```
Epoch 5/20
15/15              1s 38ms/step -
accuracy: 0.7523 - loss: 1.1516 - val_accuracy: 0.7563 - val_loss: 1.1477
Epoch 6/20
15/15              1s 39ms/step -
accuracy: 0.7738 - loss: 1.0425 - val_accuracy: 0.7629 - val_loss: 1.0916
Epoch 7/20
15/15              1s 39ms/step -
accuracy: 0.8017 - loss: 0.9370 - val_accuracy: 0.7707 - val_loss: 1.0357
Epoch 8/20
15/15              1s 36ms/step -
accuracy: 0.8114 - loss: 0.8662 - val_accuracy: 0.7752 - val_loss: 1.0045
Epoch 9/20
15/15              1s 58ms/step -
accuracy: 0.8298 - loss: 0.7921 - val_accuracy: 0.7874 - val_loss: 0.9687
Epoch 10/20
15/15              1s 42ms/step -
accuracy: 0.8365 - loss: 0.7248 - val_accuracy: 0.7924 - val_loss: 0.9501
Epoch 11/20
15/15              1s 36ms/step -
accuracy: 0.8493 - loss: 0.6815 - val_accuracy: 0.7930 - val_loss: 0.9301
Epoch 12/20
15/15              1s 33ms/step -
accuracy: 0.8596 - loss: 0.6317 - val_accuracy: 0.8036 - val_loss: 0.8970
Epoch 13/20
15/15              1s 37ms/step -
accuracy: 0.8739 - loss: 0.6030 - val_accuracy: 0.8069 - val_loss: 0.8806
Epoch 14/20
15/15              1s 35ms/step -
accuracy: 0.8849 - loss: 0.5473 - val_accuracy: 0.8075 - val_loss: 0.8625
Epoch 15/20
15/15              1s 41ms/step -
accuracy: 0.8874 - loss: 0.5208 - val_accuracy: 0.8097 - val_loss: 0.8681
Epoch 16/20
15/15              1s 35ms/step -
accuracy: 0.8885 - loss: 0.5021 - val_accuracy: 0.8130 - val_loss: 0.8545
Epoch 17/20
15/15              1s 32ms/step -
accuracy: 0.9022 - loss: 0.4589 - val_accuracy: 0.8108 - val_loss: 0.8502
Epoch 18/20
15/15              1s 33ms/step -
accuracy: 0.9062 - loss: 0.4272 - val_accuracy: 0.8097 - val_loss: 0.8341
Epoch 19/20
15/15              1s 31ms/step -
accuracy: 0.9100 - loss: 0.4104 - val_accuracy: 0.8119 - val_loss: 0.8373
Epoch 20/20
15/15              1s 31ms/step -
accuracy: 0.9150 - loss: 0.3900 - val_accuracy: 0.8080 - val_loss: 0.8280
```

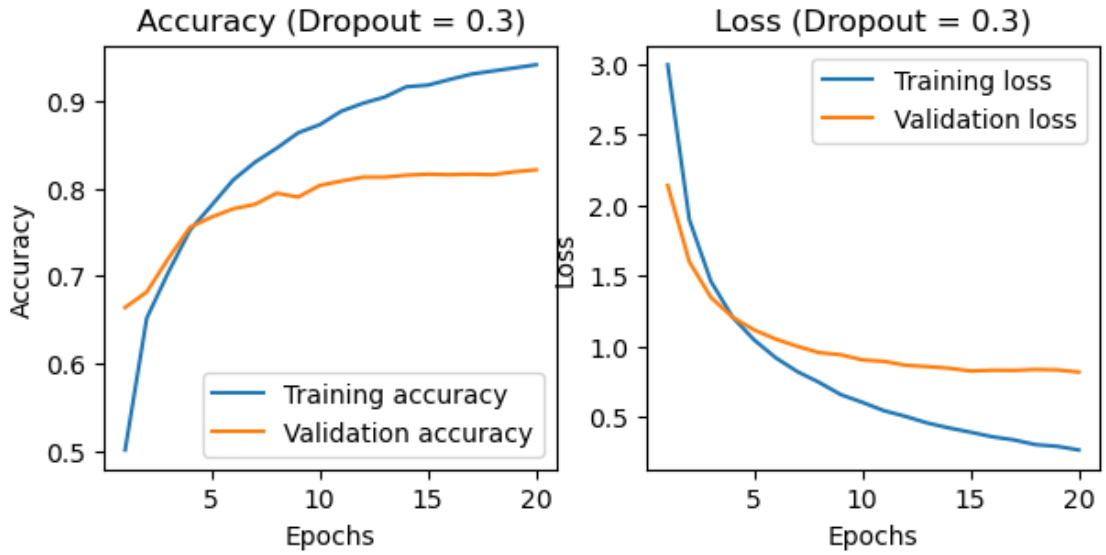
```
Training model with dropout rate = 0.8
Epoch 1/20
15/15              2s 64ms/step -
accuracy: 0.3184 - loss: 3.2094 - val_accuracy: 0.5849 - val_loss: 2.4216
Epoch 2/20
15/15              1s 32ms/step -
accuracy: 0.4899 - loss: 2.4588 - val_accuracy: 0.6411 - val_loss: 1.8854
Epoch 3/20
15/15              1s 32ms/step -
accuracy: 0.5571 - loss: 2.0801 - val_accuracy: 0.6611 - val_loss: 1.6253
Epoch 4/20
15/15              1s 30ms/step -
accuracy: 0.5901 - loss: 1.8551 - val_accuracy: 0.6817 - val_loss: 1.4567
Epoch 5/20
15/15              1s 31ms/step -
accuracy: 0.6244 - loss: 1.6586 - val_accuracy: 0.7012 - val_loss: 1.3523
Epoch 6/20
15/15              1s 32ms/step -
accuracy: 0.6395 - loss: 1.5321 - val_accuracy: 0.7145 - val_loss: 1.2791
Epoch 7/20
15/15              1s 31ms/step -
accuracy: 0.6541 - loss: 1.4602 - val_accuracy: 0.7223 - val_loss: 1.2239
Epoch 8/20
15/15              1s 41ms/step -
accuracy: 0.6693 - loss: 1.3937 - val_accuracy: 0.7307 - val_loss: 1.1911
Epoch 9/20
15/15              1s 40ms/step -
accuracy: 0.6803 - loss: 1.3316 - val_accuracy: 0.7412 - val_loss: 1.1476
Epoch 10/20
15/15              1s 34ms/step -
accuracy: 0.6952 - loss: 1.2828 - val_accuracy: 0.7412 - val_loss: 1.1289
Epoch 11/20
15/15              1s 38ms/step -
accuracy: 0.7054 - loss: 1.2232 - val_accuracy: 0.7468 - val_loss: 1.1052
Epoch 12/20
15/15              1s 43ms/step -
accuracy: 0.7090 - loss: 1.2006 - val_accuracy: 0.7568 - val_loss: 1.0851
Epoch 13/20
15/15              1s 54ms/step -
accuracy: 0.7115 - loss: 1.1536 - val_accuracy: 0.7574 - val_loss: 1.0654
Epoch 14/20
15/15              1s 46ms/step -
accuracy: 0.7197 - loss: 1.1324 - val_accuracy: 0.7635 - val_loss: 1.0438
Epoch 15/20
15/15              1s 37ms/step -
accuracy: 0.7275 - loss: 1.0813 - val_accuracy: 0.7652 - val_loss: 1.0362
Epoch 16/20
```

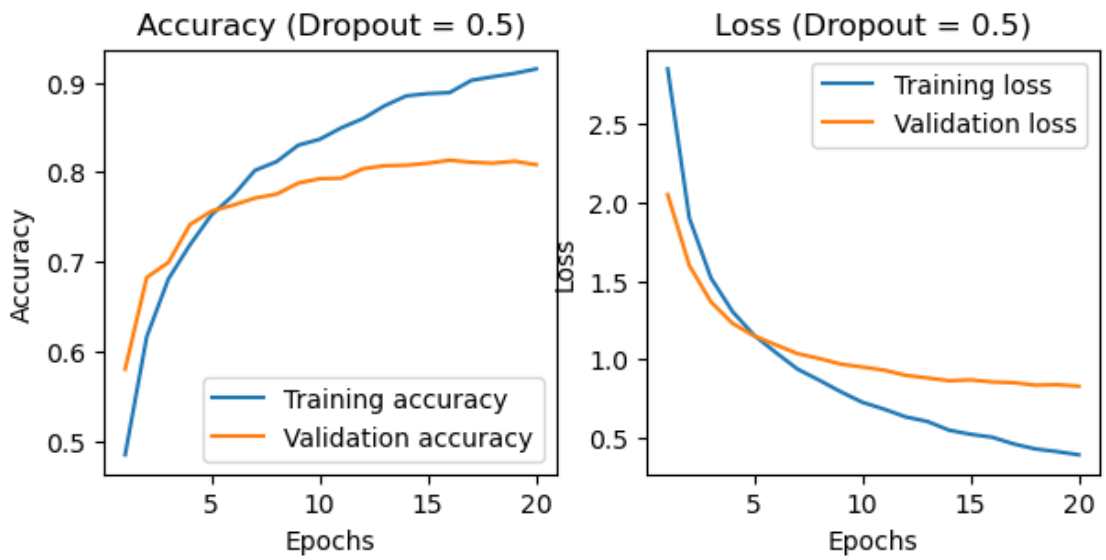
```
15/15              1s 44ms/step -
accuracy: 0.7357 - loss: 1.0736 - val_accuracy: 0.7741 - val_loss: 1.0233
Epoch 17/20
15/15              1s 57ms/step -
accuracy: 0.7397 - loss: 1.0498 - val_accuracy: 0.7780 - val_loss: 1.0089
Epoch 18/20
15/15              1s 53ms/step -
accuracy: 0.7393 - loss: 1.0335 - val_accuracy: 0.7819 - val_loss: 1.0014
Epoch 19/20
15/15              1s 60ms/step -
accuracy: 0.7467 - loss: 1.0105 - val_accuracy: 0.7802 - val_loss: 0.9925
Epoch 20/20
15/15              1s 83ms/step -
accuracy: 0.7569 - loss: 0.9520 - val_accuracy: 0.7835 - val_loss: 0.9886
```

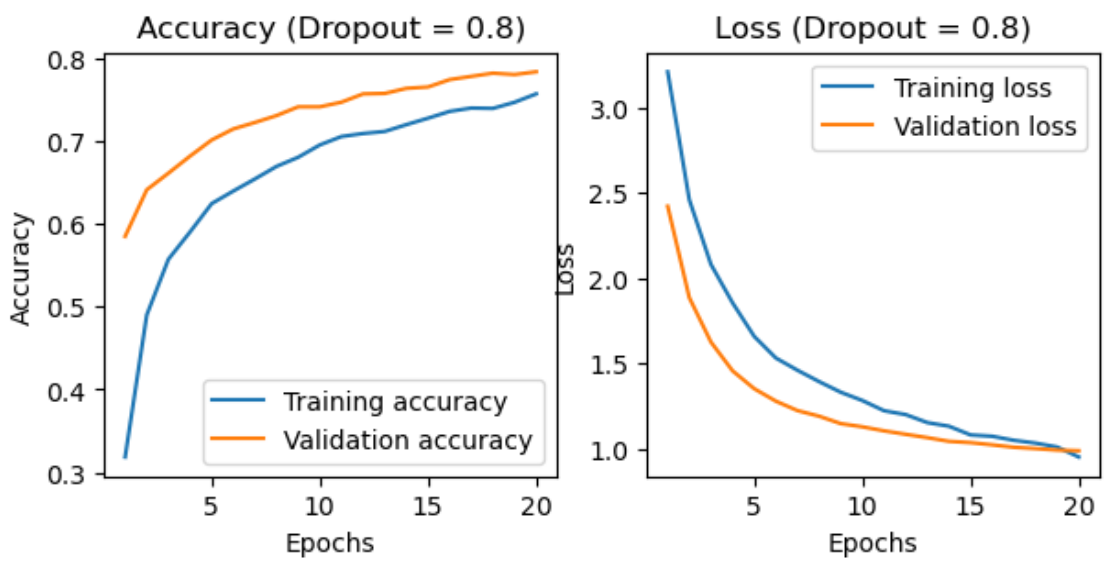
Now let's plot the graphs and evaluate

[11]:
```
for rate, history in histories_dropout.items():

    acc = history.history['accuracy']
    val_acc = history.history['val_accuracy']
    loss = history.history['loss']
    val_loss = history.history['val_loss']
    epochs = range(1, len(acc) + 1)

    fig, (ax1, ax2) = plt.subplots(1,2, figsize=(7,3))

    ax1.plot(epochs, acc, label='Training accuracy')
    ax1.plot(epochs, val_acc, label='Validation accuracy')
    ax1.set_title(f'Accuracy (Dropout = {rate})')
    ax1.set_xlabel('Epochs')
    ax1.set_ylabel('Accuracy')
    ax1.legend()

    ax2.plot(epochs, loss, label='Training loss')
    ax2.plot(epochs, val_loss, label='Validation loss')
    ax2.set_title(f'Loss (Dropout = {rate})')
    ax2.set_xlabel('Epochs')
    ax2.set_ylabel('Loss')
    ax2.legend()

    plt.show()
```


Accuracy (Dropout = 0.1)
Accuracy
Epochs
Training accuracy
Validation accuracy
Loss (Dropout = 0.1)
Loss
Epochs
Training loss
Validation loss


Accuracy (Dropout = 0.3)
Accuracy
Epochs
Training accuracy
Validation accuracy
Loss (Dropout = 0.3)
Loss
Epochs
Training loss
Validation loss




The results in the table show how different dropout levels influence the model's learning behaviour and its ability to generalise to unseen data.

When the **dropout rate is 0.1**, the model reaches the highest training accuracy at approximately **0.95**, indicating that it fits the training data very closely. Validation accuracy peaks at around **0.815**, while validation loss reaches a minimum of roughly **0.816** before increasing slightly in the final epochs. This pattern suggests that the model is learning meaningful patterns, but the relatively low level of regularisation is not sufficient to fully control overfitting. The visible gap between training and validation performance indicates that some degree of overfitting is still present.

With a **dropout rate of 0.3**, the model produces the most balanced results. Training accuracy reaches approximately **0.94**, while validation accuracy improves slightly to around **0.82**, the highest value observed among the tested configurations. Validation loss also decreases steadily during training and remains relatively stable in later epochs. This behaviour suggests that the model still has enough capacity to learn useful patterns while the regularisation helps prevent it from memorising the training data too closely. Overall, this configuration appears to offer the most effective balance between learning capacity and generalisation.

Increasing the dropout rate to **0.5** introduces stronger regularisation. Training accuracy decreases slightly to around **0.92**, while validation accuracy reaches approximately **0.81**. Validation loss also remains somewhat higher compared with the 0.3 configuration. These results indicate that although stronger dropout can reduce overfitting, it may also begin to limit the model's ability to capture the full structure of the dataset.

The configuration with a **dropout rate of 0.8** demonstrates the effects of excessive regularisation. Training accuracy only reaches about **0.75**, and validation accuracy remains below **0.79**, with validation loss staying noticeably higher throughout training. This behaviour is characteristic of **underfitting**, where too many neurons are disabled and the network struggles to learn the important patterns present in the data.

With all considered, these results highlight the trade-off between model capacity and regularisation strength. Lower dropout rates allow the network to learn more complex relationships but increase the likelihood of overfitting, while very high dropout rates restrict learning and lead to underfitting. Among the configurations tested, a **dropout rate of approximately 0.3** provides the most effective balance, achieving the highest validation accuracy while maintaining stable validation loss. For this reason, this configuration will be used as the foundation for further model tuning in the subsequent experiments.

#### 4.5.7.2 Experiment 2 - L2 Weight Regularisation

After identifying an appropriate dropout rate in the previous experiment, the next step is to examine whether additional regularisation can further improve the model's generalisation performance. One commonly used technique is **L2 weight regularisation**, which discourages the model from relying too heavily on any single connection between neurons during training. In simple terms, it penalises and discourages the model when weights become very large, encouraging the network to keep weight values smaller and learn simpler patterns from the data (1). This helps the model focus on the most meaningful relationships within the dataset rather than memorising specific details from the training data. In short, ***pattern recognition is encouraged, but only to a point.***

In this experiment, the architecture of the model will remain unchanged in order to isolate the effect of L2 weight regularisation. The hidden layer will continue to use **64 neurons with a ReLU activation function**, and the output layer will remain a **46-unit softmax layer** for multiclass classification. The dropout rate will also remain fixed at **0.3**, as this configuration produced the best balance between training and validation performance in the previous experiment. *To investigate the effect of L2 regularisation strength, several values of the L2 coefficient will be tested in separate training runs.*

The model will be trained using the same configuration as earlier experiments, including the same **number of epochs, batch size, and validation split**. Keeping these conditions constant ensures that any differences in performance can be attributed to the introduction of L2 weight regularisation rather than changes in the training procedure.

After training, the training and validation accuracy and loss will again be plotted and compared with the results from the dropout experiments. This comparison will help determine whether combining **dropout and L2 regularisation** produces more stable validation behaviour and improves the model's ability to generalise to unseen data.

[12]:
```
#L2 regularisation strengths to test
l2_values = [0, 0.00025, 0.0005, 0.001]

#store training histories
histories_l2 = {}

for value in l2_values:

    print(f"\nTraining model with L2 regularisation = {value}")

    #construct model
    model_l2 = keras.Sequential()

    #hidden layer with L2 penalty added
    model_l2.add(
        layers.Dense(
            64,
            activation='relu',
            input_shape=(10000,),
            kernel_regularizer=regularizers.l2(value)
        )
    )

    #best dropout rate from previous experiment
    model_l2.add(layers.Dropout(0.3))

    #unchanged output layer
    model_l2.add(layers.Dense(46, activation='softmax'))

    #unchanged compile model
    model_l2.compile(
        optimizer='rmsprop',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    #unchanged train model
    history = model_l2.fit(
        x_train,
        y_train,
        epochs=20,
        batch_size=512,
```

```
        validation_split=0.2
    )

    histories_l2[value] = history
```

```
Training model with L2 regularisation = 0
Epoch 1/20
15/15              4s 99ms/step -
accuracy: 0.5211 - loss: 2.7909 - val_accuracy: 0.6494 - val_loss: 1.9900
Epoch 2/20
15/15              1s 39ms/step -
accuracy: 0.6537 - loss: 1.7848 - val_accuracy: 0.6884 - val_loss: 1.5373
Epoch 3/20
15/15              1s 35ms/step -
accuracy: 0.7045 - loss: 1.3890 - val_accuracy: 0.7184 - val_loss: 1.3187
Epoch 4/20
15/15              1s 46ms/step -
accuracy: 0.7513 - loss: 1.1616 - val_accuracy: 0.7396 - val_loss: 1.1999
Epoch 5/20
15/15              1s 39ms/step -
accuracy: 0.7822 - loss: 1.0143 - val_accuracy: 0.7535 - val_loss: 1.1216
Epoch 6/20
15/15              1s 36ms/step -
accuracy: 0.8056 - loss: 0.8965 - val_accuracy: 0.7646 - val_loss: 1.0575
Epoch 7/20
15/15              1s 38ms/step -
accuracy: 0.8310 - loss: 0.8027 - val_accuracy: 0.7813 - val_loss: 1.0063
Epoch 8/20
15/15              1s 37ms/step -
accuracy: 0.8459 - loss: 0.7250 - val_accuracy: 0.7880 - val_loss: 0.9695
Epoch 9/20
15/15              1s 39ms/step -
accuracy: 0.8635 - loss: 0.6594 - val_accuracy: 0.7902 - val_loss: 0.9489
Epoch 10/20
15/15              1s 40ms/step -
accuracy: 0.8750 - loss: 0.5987 - val_accuracy: 0.7991 - val_loss: 0.9107
Epoch 11/20
15/15              1s 35ms/step -
accuracy: 0.8885 - loss: 0.5446 - val_accuracy: 0.8091 - val_loss: 0.8834
Epoch 12/20
15/15              1s 32ms/step -
accuracy: 0.9022 - loss: 0.4982 - val_accuracy: 0.8097 - val_loss: 0.8704
Epoch 13/20
15/15              1s 39ms/step -
accuracy: 0.9095 - loss: 0.4565 - val_accuracy: 0.8125 - val_loss: 0.8644
Epoch 14/20
15/15              1s 39ms/step -
```

```
accuracy: 0.9151 - loss: 0.4172 - val_accuracy: 0.8097 - val_loss: 0.8632
Epoch 15/20
15/15            1s 36ms/step -
accuracy: 0.9198 - loss: 0.3894 - val_accuracy: 0.8141 - val_loss: 0.8451
Epoch 16/20
15/15            1s 37ms/step -
accuracy: 0.9294 - loss: 0.3566 - val_accuracy: 0.8197 - val_loss: 0.8289
Epoch 17/20
15/15            1s 37ms/step -
accuracy: 0.9340 - loss: 0.3255 - val_accuracy: 0.8175 - val_loss: 0.8353
Epoch 18/20
15/15            1s 36ms/step -
accuracy: 0.9333 - loss: 0.3109 - val_accuracy: 0.8152 - val_loss: 0.8286
Epoch 19/20
15/15            1s 34ms/step -
accuracy: 0.9390 - loss: 0.2854 - val_accuracy: 0.8152 - val_loss: 0.8346
Epoch 20/20
15/15            1s 35ms/step -
accuracy: 0.9443 - loss: 0.2668 - val_accuracy: 0.8180 - val_loss: 0.8195

Training model with L2 regularisation = 0.00025
Epoch 1/20
15/15            7s 72ms/step -
accuracy: 0.5228 - loss: 2.7628 - val_accuracy: 0.6433 - val_loss: 2.0179
Epoch 2/20
15/15            1s 34ms/step -
accuracy: 0.6572 - loss: 1.8083 - val_accuracy: 0.7112 - val_loss: 1.5797
Epoch 3/20
15/15            1s 32ms/step -
accuracy: 0.7145 - loss: 1.4306 - val_accuracy: 0.7212 - val_loss: 1.3663
Epoch 4/20
15/15            1s 39ms/step -
accuracy: 0.7545 - loss: 1.2147 - val_accuracy: 0.7446 - val_loss: 1.2402
Epoch 5/20
15/15            1s 33ms/step -
accuracy: 0.7829 - loss: 1.0750 - val_accuracy: 0.7568 - val_loss: 1.1581
Epoch 6/20
15/15            1s 39ms/step -
accuracy: 0.8049 - loss: 0.9528 - val_accuracy: 0.7724 - val_loss: 1.1033
Epoch 7/20
15/15            1s 41ms/step -
accuracy: 0.8302 - loss: 0.8719 - val_accuracy: 0.7835 - val_loss: 1.0521
Epoch 8/20
15/15            1s 41ms/step -
accuracy: 0.8484 - loss: 0.7857 - val_accuracy: 0.7924 - val_loss: 1.0198
Epoch 9/20
15/15            1s 41ms/step -
accuracy: 0.8628 - loss: 0.7237 - val_accuracy: 0.8002 - val_loss: 0.9895
```

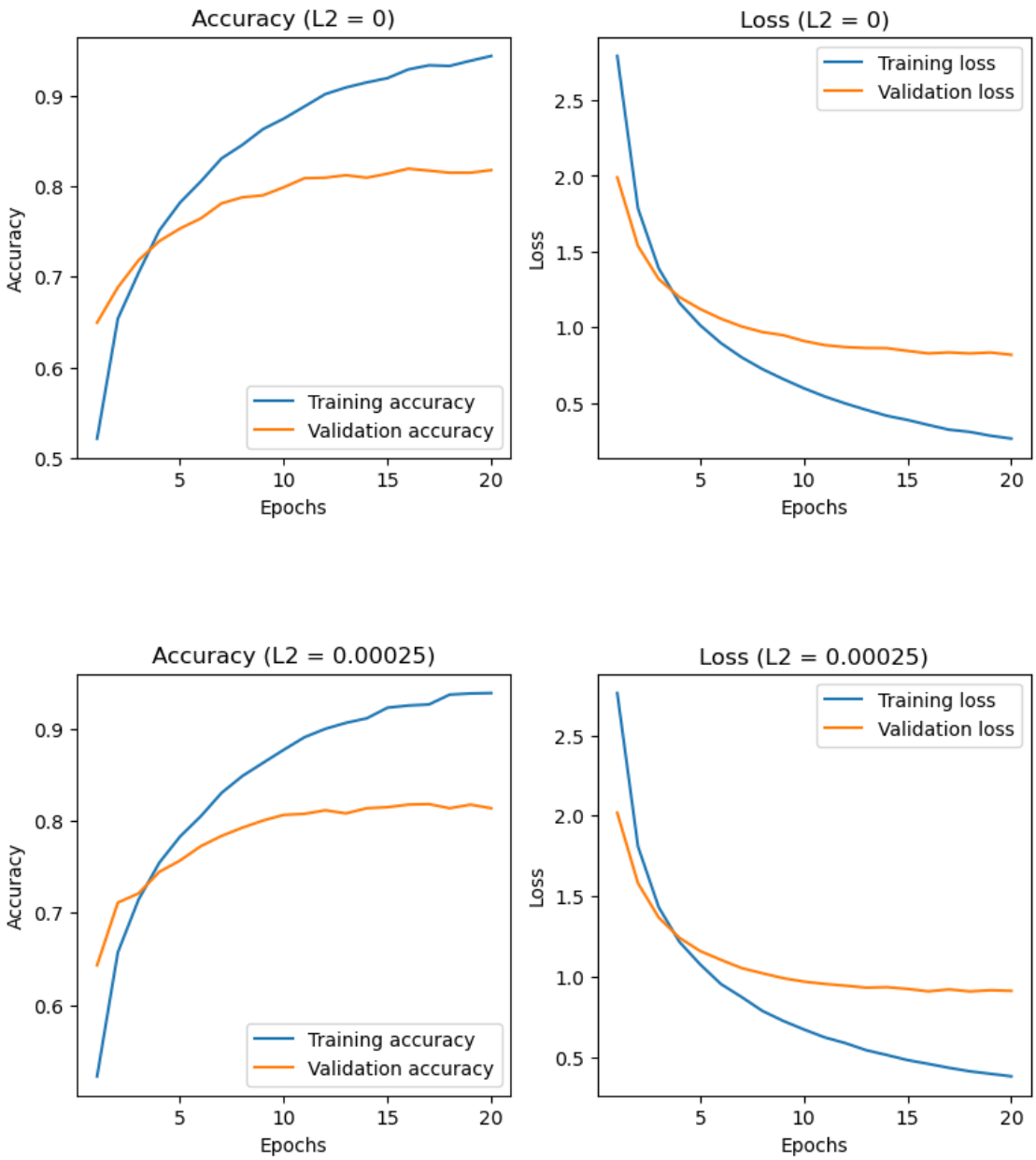
```
Epoch 10/20
15/15              1s 44ms/step -
accuracy: 0.8770 - loss: 0.6705 - val_accuracy: 0.8063 - val_loss: 0.9680
Epoch 11/20
15/15              1s 42ms/step -
accuracy: 0.8906 - loss: 0.6213 - val_accuracy: 0.8075 - val_loss: 0.9537
Epoch 12/20
15/15              1s 51ms/step -
accuracy: 0.8997 - loss: 0.5854 - val_accuracy: 0.8114 - val_loss: 0.9429
Epoch 13/20
15/15              1s 55ms/step -
accuracy: 0.9062 - loss: 0.5416 - val_accuracy: 0.8080 - val_loss: 0.9304
Epoch 14/20
15/15              1s 37ms/step -
accuracy: 0.9111 - loss: 0.5115 - val_accuracy: 0.8136 - val_loss: 0.9334
Epoch 15/20
15/15              1s 41ms/step -
accuracy: 0.9226 - loss: 0.4797 - val_accuracy: 0.8147 - val_loss: 0.9226
Epoch 16/20
15/15              1s 38ms/step -
accuracy: 0.9250 - loss: 0.4564 - val_accuracy: 0.8175 - val_loss: 0.9075
Epoch 17/20
15/15              1s 32ms/step -
accuracy: 0.9261 - loss: 0.4317 - val_accuracy: 0.8180 - val_loss: 0.9197
Epoch 18/20
15/15              1s 37ms/step -
accuracy: 0.9368 - loss: 0.4100 - val_accuracy: 0.8136 - val_loss: 0.9070
Epoch 19/20
15/15              1s 36ms/step -
accuracy: 0.9381 - loss: 0.3945 - val_accuracy: 0.8175 - val_loss: 0.9144
Epoch 20/20
15/15              1s 34ms/step -
accuracy: 0.9385 - loss: 0.3789 - val_accuracy: 0.8136 - val_loss: 0.9105

Training model with L2 regularisation = 0.0005
Epoch 1/20
15/15              3s 76ms/step -
accuracy: 0.4960 - loss: 2.9169 - val_accuracy: 0.5927 - val_loss: 2.1583
Epoch 2/20
15/15              1s 33ms/step -
accuracy: 0.6373 - loss: 1.9175 - val_accuracy: 0.6917 - val_loss: 1.6632
Epoch 3/20
15/15              1s 37ms/step -
accuracy: 0.7079 - loss: 1.5073 - val_accuracy: 0.7056 - val_loss: 1.4319
Epoch 4/20
15/15              1s 36ms/step -
accuracy: 0.7443 - loss: 1.2747 - val_accuracy: 0.7373 - val_loss: 1.2840
Epoch 5/20
```

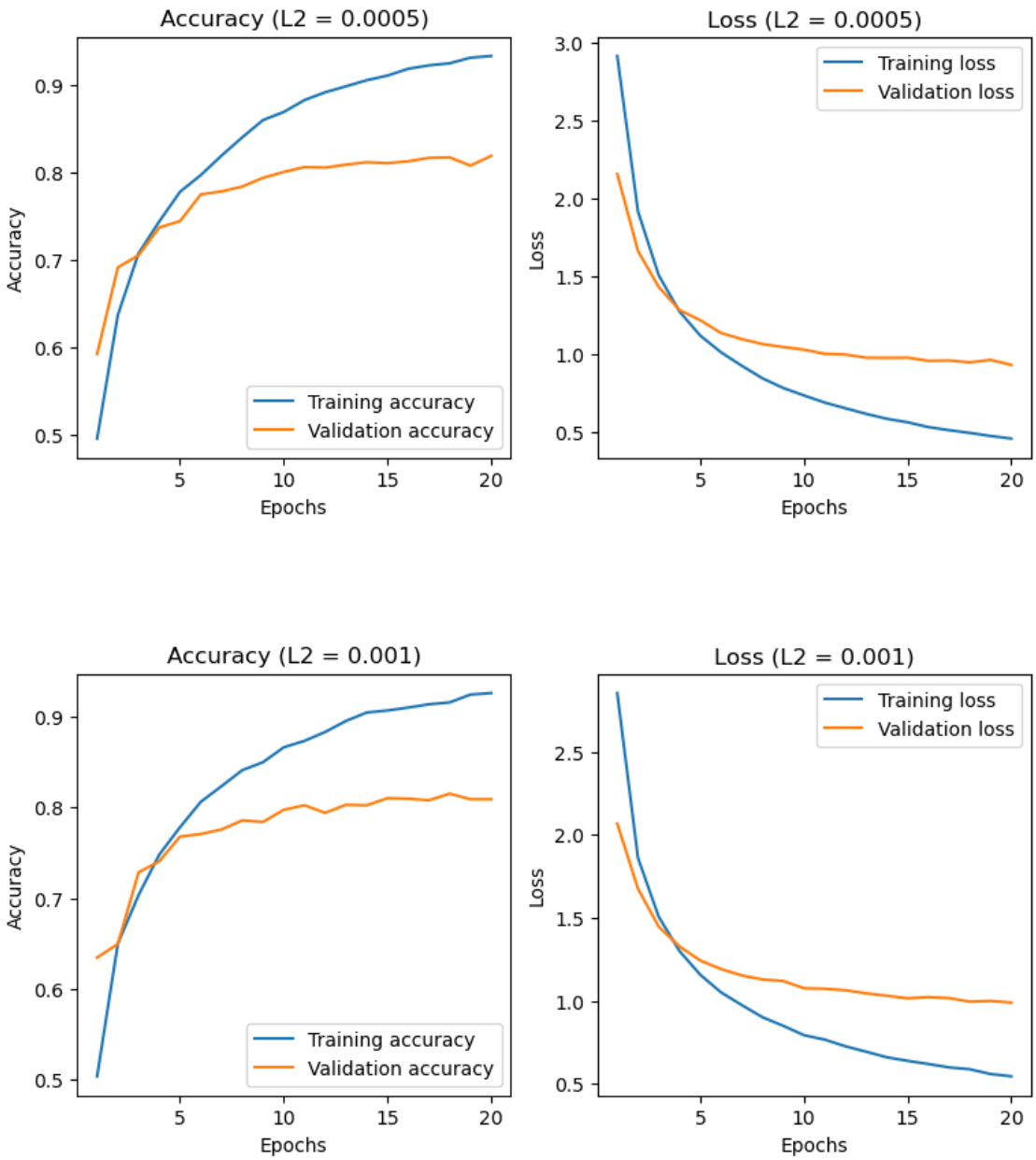
```
15/15              1s 39ms/step -
accuracy: 0.7781 - loss: 1.1201 - val_accuracy: 0.7446 - val_loss: 1.2191
Epoch 6/20
15/15              1s 37ms/step -
accuracy: 0.7972 - loss: 1.0126 - val_accuracy: 0.7752 - val_loss: 1.1368
Epoch 7/20
15/15              1s 36ms/step -
accuracy: 0.8195 - loss: 0.9262 - val_accuracy: 0.7785 - val_loss: 1.0974
Epoch 8/20
15/15              1s 38ms/step -
accuracy: 0.8404 - loss: 0.8453 - val_accuracy: 0.7841 - val_loss: 1.0655
Epoch 9/20
15/15              1s 38ms/step -
accuracy: 0.8601 - loss: 0.7832 - val_accuracy: 0.7941 - val_loss: 1.0466
Epoch 10/20
15/15              1s 44ms/step -
accuracy: 0.8693 - loss: 0.7353 - val_accuracy: 0.8008 - val_loss: 1.0294
Epoch 11/20
15/15              1s 36ms/step -
accuracy: 0.8830 - loss: 0.6894 - val_accuracy: 0.8063 - val_loss: 1.0029
Epoch 12/20
15/15              1s 39ms/step -
accuracy: 0.8919 - loss: 0.6527 - val_accuracy: 0.8058 - val_loss: 0.9981
Epoch 13/20
15/15              1s 34ms/step -
accuracy: 0.8987 - loss: 0.6169 - val_accuracy: 0.8091 - val_loss: 0.9779
Epoch 14/20
15/15              1s 35ms/step -
accuracy: 0.9056 - loss: 0.5851 - val_accuracy: 0.8119 - val_loss: 0.9770
Epoch 15/20
15/15              1s 38ms/step -
accuracy: 0.9109 - loss: 0.5627 - val_accuracy: 0.8108 - val_loss: 0.9781
Epoch 16/20
15/15              1s 34ms/step -
accuracy: 0.9187 - loss: 0.5317 - val_accuracy: 0.8130 - val_loss: 0.9575
Epoch 17/20
15/15              1s 38ms/step -
accuracy: 0.9226 - loss: 0.5118 - val_accuracy: 0.8169 - val_loss: 0.9597
Epoch 18/20
15/15              1s 32ms/step -
accuracy: 0.9250 - loss: 0.4943 - val_accuracy: 0.8175 - val_loss: 0.9479
Epoch 19/20
15/15              1s 33ms/step -
accuracy: 0.9314 - loss: 0.4744 - val_accuracy: 0.8080 - val_loss: 0.9638
Epoch 20/20
15/15              1s 38ms/step -
accuracy: 0.9333 - loss: 0.4579 - val_accuracy: 0.8191 - val_loss: 0.9316
```

```
Training model with L2 regularisation = 0.001
Epoch 1/20
15/15              2s 82ms/step -
accuracy: 0.5034 - loss: 2.8547 - val_accuracy: 0.6344 - val_loss: 2.0685
Epoch 2/20
15/15              1s 50ms/step -
accuracy: 0.6497 - loss: 1.8610 - val_accuracy: 0.6494 - val_loss: 1.6753
Epoch 3/20
15/15              1s 41ms/step -
accuracy: 0.7034 - loss: 1.5067 - val_accuracy: 0.7284 - val_loss: 1.4468
Epoch 4/20
15/15              1s 39ms/step -
accuracy: 0.7481 - loss: 1.3005 - val_accuracy: 0.7407 - val_loss: 1.3280
Epoch 5/20
15/15              1s 46ms/step -
accuracy: 0.7783 - loss: 1.1576 - val_accuracy: 0.7679 - val_loss: 1.2436
Epoch 6/20
15/15              1s 44ms/step -
accuracy: 0.8063 - loss: 1.0516 - val_accuracy: 0.7707 - val_loss: 1.1919
Epoch 7/20
15/15              1s 34ms/step -
accuracy: 0.8235 - loss: 0.9751 - val_accuracy: 0.7757 - val_loss: 1.1536
Epoch 8/20
15/15              1s 39ms/step -
accuracy: 0.8412 - loss: 0.9017 - val_accuracy: 0.7858 - val_loss: 1.1294
Epoch 9/20
15/15              1s 41ms/step -
accuracy: 0.8500 - loss: 0.8500 - val_accuracy: 0.7841 - val_loss: 1.1197
Epoch 10/20
15/15              1s 42ms/step -
accuracy: 0.8664 - loss: 0.7935 - val_accuracy: 0.7974 - val_loss: 1.0764
Epoch 11/20
15/15              1s 76ms/step -
accuracy: 0.8735 - loss: 0.7668 - val_accuracy: 0.8024 - val_loss: 1.0730
Epoch 12/20
15/15              1s 37ms/step -
accuracy: 0.8834 - loss: 0.7264 - val_accuracy: 0.7941 - val_loss: 1.0641
Epoch 13/20
15/15              1s 38ms/step -
accuracy: 0.8955 - loss: 0.6938 - val_accuracy: 0.8030 - val_loss: 1.0457
Epoch 14/20
15/15              1s 38ms/step -
accuracy: 0.9048 - loss: 0.6600 - val_accuracy: 0.8024 - val_loss: 1.0311
Epoch 15/20
15/15              1s 36ms/step -
accuracy: 0.9070 - loss: 0.6388 - val_accuracy: 0.8102 - val_loss: 1.0156
Epoch 16/20
15/15              1s 36ms/step -
```

```
accuracy: 0.9102 - loss: 0.6198 - val_accuracy: 0.8097 - val_loss: 1.0229
Epoch 17/20
15/15              1s 38ms/step -
accuracy: 0.9138 - loss: 0.5992 - val_accuracy: 0.8080 - val_loss: 1.0167
Epoch 18/20
15/15              1s 48ms/step -
accuracy: 0.9159 - loss: 0.5880 - val_accuracy: 0.8152 - val_loss: 0.9951
Epoch 19/20
15/15              1s 45ms/step -
accuracy: 0.9246 - loss: 0.5593 - val_accuracy: 0.8091 - val_loss: 0.9993
Epoch 20/20
15/15              1s 41ms/step -
accuracy: 0.9262 - loss: 0.5455 - val_accuracy: 0.8091 - val_loss: 0.9899
```

[13]:
```python
for value, history in histories_l2.items():

    acc = history.history['accuracy']
    val_acc = history.history['val_accuracy']
    loss = history.history['loss']
    val_loss = history.history['val_loss']

    epochs = range(1, len(acc) + 1)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9,4))

    #accuracy plot
    ax1.plot(epochs, acc, label='Training accuracy')
    ax1.plot(epochs, val_acc, label='Validation accuracy')
    ax1.set_title(f'Accuracy (L2 = {value})')
    ax1.set_xlabel('Epochs')
    ax1.set_ylabel('Accuracy')
    ax1.legend()

    #loss plot
    ax2.plot(epochs, loss, label='Training loss')
    ax2.plot(epochs, val_loss, label='Validation loss')
    ax2.set_title(f'Loss (L2 = {value})')
    ax2.set_xlabel('Epochs')
    ax2.set_ylabel('Loss')
    ax2.legend()

    plt.show()
```


Accuracy (L2 = 0)
Accuracy
Epochs
Training accuracy
Validation accuracy
Loss (L2 = 0)
Loss
Epochs
Training loss
Validation loss
Accuracy (L2 = 0.00025)
Accuracy
Epochs
Training accuracy
Validation accuracy
Loss (L2 = 0.00025)
Loss
Epochs
Training loss
Validation loss




Initially, three different values were selected for the L2 weight regularisation experiment. However, the smallest value produced the strongest validation results. When a new, smaller value was introduced, that again produced the best performance. This pattern suggested that the optimal value might be even closer to zero. To investigate this further, the experiment was repeated a third time with **no L2 regularisation** applied, allowing the model to be compared directly with a version that relied only on dropout for regularisation.

Including this additional run provides a clearer view of how the different L2 strengths influence model behaviour. Overall, the results suggest that **very small L2 penalties do not significantly improve performance when dropout is already present**, and in some cases may slightly limit

the model's ability to learn.

When **no L2 regularisation** is applied, the model performs strongly. Training accuracy increases steadily throughout the epochs and finishes at approximately **0.94**, while validation accuracy reaches around **0.82**. Validation loss ends at roughly **0.82**, which is the lowest value observed among the tested configurations. This demonstrates that the model is able to learn meaningful patterns from the training data while still maintaining stable performance on unseen validation samples.

Introducing a small penalty with **L2 = 0.00025** produces very similar behaviour, but with slightly weaker validation performance. Training accuracy again approaches **0.94**, but validation accuracy settles closer to **0.81**, and validation loss remains marginally higher than in the configuration without L2 suggesting that the additional constraint on the weights does not provide a clear improvement in generalisation and may slightly restrict the optimisation process.

When the regularisation strength is increased to **0.0005**, this effect becomes more noticeable. Although training accuracy still reaches a high level, validation accuracy does not improve and remains around **0.81**, while validation loss is consistently higher than the baseline model. This shows that the stronger penalty begins to limit the model's ability to capture more detailed relationships within the dataset.

**L2 = 0.001** leads to the worst relative performance. While the model still learns effectively, it seems training accuracy increases more slowly and validation loss remains higher throughout the training process. This pattern suggests that the stronger penalty constrains the network's weights too heavily, reducing its capacity to represent the structure of the data.

Taken together, these results indicate that **adding L2 regularisation does not provide a clear benefit for this model when dropout is already being used**. In fact, the configuration without L2 regularisation produced the best overall model performance. This suggests that the dropout layer introduced earlier already provides sufficient regularisation, and that adding further penalties to the weights may unnecessarily restrict the model's learning capacity and add no benefit whatsoever.

#### 4.5.7.3 - Conclusion

This section focused on improving the model by experimenting with different regularisation techniques and observing how they affected performance on unseen data. The aim was to find a configuration that allows the model to learn useful patterns from the training data without simply memorising it.

The first experiment explored different **dropout rates**. The results showed that the amount of dropout has a noticeable impact on how the model learns. When the dropout value was too high, too many neurons were switched off during training and the model struggled to learn effectively. When the value was too low, the model fitted the training data very closely but the gap between training and validation performance increased. Overall, a **dropout rate of 0.3** provided the most balanced results, allowing the model to learn efficiently while still maintaining stable validation performance.

The second experiment examined whether adding **L2 weight regularisation** could further improve generalisation. Several L2 values were tested, gradually reducing the strength of the penalty applied to the weights. Interestingly, the results showed that adding L2 regularisation did not improve performance compared with the model that used dropout alone. In fact, as the regularisation

strength increased, the model's ability to learn the underlying structure of the dataset became slightly more limited.

Based on these experiments, the most effective configuration for this task is the model that uses **dropout with a rate of 0.3 and no additional L2 regularisation**. This setup produced the strongest validation performance while maintaining stable learning behaviour. Overall, the tuning process highlights how small changes to regularisation techniques can influence model behaviour, and demonstrates the importance of testing different configurations in order to find a good balance between learning the training data and generalising to new examples.

**References**


(1) Chollet, F., 2018. Deep Learning with Python. 1st ed. Shelter Island, NY: Manning Publications.

(2) TensorFlow, Reuters dataset – Available at: https://www.tensorflow.org/api_docs/python/tf/keras/datasets (Accessed: 1 March 2026).

(3) TensorFlow, Text classification with TensorFlow and Keras. Available at: https://www.tensorflow.org/tutorials/keras/text_classification (Accessed: 1 March 2026).

(4) TensorFlow, Classification metrics. Available at: https://www.tensorflow.org (Accessed: 1 March 2026).

(5) Sentdex, Neural Networks from Scratch. Available at: https://www.youtube.com/watch?v=dEXPMQXoiLc (Accessed: 2 March 2026)

(6) TensorFlow, Training and evaluation in Keras. Available at: https://www.tensorflow.org/guide/keras/training_with_built_in_methods (Accessed: 2 March 2026).

(7) Chandan, G., 2022. Training, Testing, Validation Datasets and Cross-Validation. Available at: https://gschandan.dev/posts/2022-04-27-testing-validation.html (Accessed: 3 March 2026).

(8) TensorFlow, 2024. Keras Sequential model documentation. Available at: https://keras.io/api/models/sequential/ (Accessed: 3 March 2026).

(9) TensorFlow, Dropout layer. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dro (Accessed: 3 March 2026)


[ ]:

```
```